

Project 1:Spartan6 - DSP48A1

NAME: kerolos Nasser Adly

1:RTL :

1-DSP.V

```
module DSP(A,B,C,D,
           clk,CARRYIN,
           OPMODE,BCIN,RSTA,RSTB,RSTC,RSTD,RSTP,RSTM,RSTCARRYIN,RSTOPMODE,
           CEA,CEB,CEC,CED,CEP,CEM,CECARRYIN,CEOPMODE,
           PCIN,BCOUT,POUT,
           P,M,CARRYOUT,CARRYOUTF);

/*
parameters for DSP48A1
*/
parameter A0REG=0;
parameter A1REG=1;
parameter B0REG=0;
parameter B1REG=1;

parameter CREG=1;
parameter DREG=1;
parameter MREG=1;
parameter PREG=1;
parameter CARRYINREG=1;
parameter CARRYOUTREG=1;
parameter OPMODEREG=1;
parameter CARRYINSEL="OPMODE[5]"; //or "CARRYIN"
parameter B_INPUT="DIRECT"; //or "CASCADE"
parameter RSTTYPE="SYNC"; //or "ASYNC"

/*
Inputs for DSP48A1
*/
input [17:0] A;
input [17:0] B;
input [17:0] BCIN;
input [47:0] C;
input [17:0] D;
input CARRYIN;
input clk;
input [7:0] OPMODE;
input CEA,CEB,CEC,CED,CEP,CEM,CECARRYIN,CEOPMODE;
input RSTA,RSTB,RSTC,RSTD,RSTP,RSTM,RSTCARRYIN,RSTOPMODE;
input [47:0] PCIN;

/*
Outputs for DSP48A1
*/
output [35:0] M;
output [47:0] P;
output CARRYOUT;
output CARRYOUTF;
output [47:0] POUT;
output [17:0] BCOUT;
```

```

//PART1: REGs for A,B,C,D,OPMODE
//Opmode reg
wire [7:0] OPMODE_reg;
REG_pipe #(.width(8),.REG_EN(OPMODEREG),.REG_type(RSTTYPE))
OPMODE_REG(.D(OPMODE),.Q(OPMODE_reg),.rst(RSTOPMODE),.clk(clk),.CE(CEOPMODE));

//B input mux
wire [17:0] B_Cascade;
generate
    if(B_INPUT=="DIRECT") begin
        assign B_Cascade = B;
    end
    else if(B_INPUT=="CASCADE") begin
        assign B_Cascade = BCIN;
    end
    else begin
        assign B_Cascade = 18'b0;
    end
endgenerate

//Regs for A,B,C,D
wire [17:0] A0_reg,B0_reg,D0_reg;
wire [47:0] C0_reg;

REG_pipe #(.width(18),.REG_EN(A0REG),.REG_type(RSTTYPE))
A0_REG(.D(A),.Q(A0_reg),.rst(RSTA),.clk(clk),.CE(CEA));
REG_pipe #(.width(18),.REG_EN(B0REG),.REG_type(RSTTYPE))
B0_REG(.D(B_Cascade),.Q(B0_reg),.rst(RSTB),.clk(clk),.CE(CEB));
REG_pipe #(.width(18),.REG_EN(DREG),.REG_type(RSTTYPE))
D0_REG(.D(D),.Q(D0_reg),.rst(RSTD),.clk(clk),.CE(CED));
REG_pipe #(.width(48),.REG_EN(CREG),.REG_type(RSTTYPE))
C0_REG(.D(C),.Q(C0_reg),.rst(RSTC),.clk(clk),.CE(CEC));

//Pre_ADDER/SUBTRACTOR
wire [17:0] pre_adder_out;
assign pre_adder_out = OPMODE_reg[6] ? (D0_reg - B0_reg) : (D0_reg + B0_reg);

wire [17:0] pre_adder_out_mux;
assign pre_adder_out_mux = OPMODE_reg[4] ? pre_adder_out:B0_reg;

//PART2: MULTIPLIER
wire [17:0] B1_reg;
wire [17:0] A1_reg;

REG_pipe #(.width(18),.REG_EN(B1REG),.REG_type(RSTTYPE))
B1_REG(.D(pre_adder_out_mux),.Q(B1_reg),.rst(RSTB),.clk(clk),.CE(CEB));
REG_pipe #(.width(18),.REG_EN(A1REG),.REG_type(RSTTYPE))
A1_REG(.D(A0_reg),.Q(A1_reg),.rst(RSTA),.clk(clk),.CE(CEA));
assign BCOUT = B1_reg;

wire [35:0] M_reg;
wire [35:0] M_out;
assign M_reg = A1_reg * B1_reg;

REG_pipe #(.width(36),.REG_EN(MREG),.REG_type(RSTTYPE))
M_REG(.D(M_reg),.Q(M_out),.rst(RSTM),.clk(clk),.CE(CEM));
assign M = M_out;

//PART3: POST_ADDER/SUBTRACTOR
//D:A:B Concatenation
wire [47:0] DAB_concat;
assign DAB_concat = {D0_reg[11:0],A1_reg[17:0],B1_reg[17:0]};

//mux X
reg [47:0] mux_X_out;

```

```

always@(*) begin
    case(OPMODE_reg[1:0])

        2'b00: mux_X_out = 48'b0;
        2'b01: mux_X_out = {12'b0,M};
        2'b10: mux_X_out = P;
        2'b11: mux_X_out = DAB_concat;
        default: mux_X_out = 48'b0;

    endcase

end

//mux Z
reg [47:0] mux_Z_out;
always@(*) begin
    case(OPMODE_reg[3:2])

        2'b00: mux_Z_out = 48'b0;
        2'b01: mux_Z_out = PCIN;
        2'b10: mux_Z_out = P;
        2'b11: mux_Z_out = C0_reg;

        default: mux_Z_out = 48'b0;

    endcase

end

//Carry in for post adder/subtractor
wire post_adder_carry_in_mux;
generate
    if(CARRYINSEL=="OPMODE[5]") begin
        assign post_adder_carry_in_mux = OPMODE_reg[5];
    end
    else if(CARRYINSEL=="CARRYIN") begin
        assign post_adder_carry_in_mux = CARRYIN;
    end
endgenerate

wire CIN_reg;
REG_pipe #(.width(1),.REG_EN(CARRYINREG),.REG_type(RSTTYPE))
POST_ADDER_CARRY_IN_REG(.D(post_adder_carry_in_mux),.Q(CIN_reg),.rst(RSTCARRYIN),.clk(clk),.CE(CECARRYIN));

//POST adder/subtractor
wire [47:0] post_adder_out;
wire carry_out_from_post_adder;
assign {carry_out_from_post_adder,post_adder_out} = OPMODE_reg[7] ? (mux_Z_out - (mux_X_out + CIN_reg)) : (mux_Z_out + (mux_X_out + CIN_reg));
//Carry out reg
wire carry_out_reg;
REG_pipe #(.width(1),.REG_EN(CARRYOUTREG),.REG_type(RSTTYPE))
CARRYOUT_REG(.D(carry_out_from_post_adder),.Q(carry_out_reg),.rst(RSTCARRYIN),.clk(clk),.CE(CECARRYIN));
assign CARRYOUT = carry_out_reg;
assign CARRYOUTF = carry_out_from_post_adder;
//P reg
wire [47:0] P_reg;
REG_pipe #(.width(48),.REG_EN(PREG),.REG_type(RSTTYPE))
P_REG(.D(post_adder_out),.Q(P_reg),.rst(RSTP),.clk(clk),.CE(CEP));
assign P = P_reg;
assign POUT = P_reg;

endmodule

```

2-REG_pipe.v

```
module REG_pipe(D,Q,rst,clk,CE);
parameter width=18;
parameter REG_EN=1;
parameter REG_type="SYNC";

input [width-1:0] D;
output reg [width-1:0] Q;
input rst;
input clk;
input CE;

generate
    if(REG_EN) begin
        if(REG_type=="SYNC") begin
            always@(posedge clk) begin
                if (rst) begin
                    Q <= 0;
                end else if (CE) begin
                    Q <= D;
                end
            end
        end

        else if(REG_type=="ASYNC") begin
            always@(posedge clk or posedge rst) begin
                if (rst) begin
                    Q <= 0;
                end else if (CE) begin
                    Q <= D;
                end
            end
        end
    end
else begin
    //no reg
    always@(*) begin
        Q = D;
    end
end
endgenerate
endmodule
```

2-TEST BENCH

```
module DSP_tb;

// signal declaration
reg [17:0] A,B,D;
reg [17:0] BCIN;
reg [47:0] C,PCIN;
reg [7:0] OPMODE;
reg CARRYIN;
reg clk;
reg CEA,CEB,CEC,CED,CEP,CEM,CECARRYIN,CEOPMODE;
reg RSTA,RSTB,RSTC,RSTD,RSTP,RSTM,RSTCARRYIN,RSTOPMODE;

wire [35:0] M;
wire [47:0] P;
wire CARRYOUT,CARRYOUTF;
wire [47:0] POUT;
wire [17:0] BCOUT;

// Instantiate the DSP module
DSP DUT(.A(A),.B(B),.C(C),.D(D),
        .clk(clk),.CARRYIN(CARRYIN),
        .OPMODE(OPMODE),.BCIN(BCIN),.RSTA(RSTA),.RSTB(RSTB),.RSTC(RSTC),
        .RSTD(RSTD),.RSTP(RSTP),.RSTM(RSTM),.RSTCARRYIN(RSTCARRYIN),.RSTOPMODE(RSTOPMODE),
        .CEA(CEA),.CEB(CEB),.CEC(CEC),.CED(CED),.CEP(CEP),.CEM(CEM),.CECARRYIN(CECARRYIN),.CEOPMODE(CEOPMODE),
        .PCIN(PCIN),
        .BCOUT(BCOUT),
        .POUT(POUT),
        .P(P),
        .M(M),
        .CARRYOUT(CARRYOUT),
        .CARRYOUTF(CARRYOUTF)
);

// Clock generation
initial begin
    clk = 0;
    forever #5 clk = ~clk; // 100MHz clock
end

// Test stimulus
initial begin
```

```

//reset operations
RSTA = 1; RSTB = 1; RSTC = 1; RSTD = 1; RSTP = 1; RSTM = 1; RSTCARRYIN
= 1; RSTOPMODE = 1;

CEA = $random;
CEB = $random;
CEC = $random;
CED = $random;
CEP = $random;
CEM = $random;
CECARRYIN = $random;
CEOPMODE = $random;

A=$random;
B=$random;
C=$random;
D=$random;
BCIN=$random;
OPMODE=$random;
CARRYIN=$random;
PCIN=$random;

// give reset time for pipeline
repeat(3) @(negedge clk);

if(M !== 36'b0 || P !== 48'b0 || CARRYOUT !== 1'b0 || CARRYOUTF !==
1'b0 || POUT !== 48'b0 || BCOUT !== 18'b0) begin
    $display("Reset failed!");
end else begin
    $display("Reset successful.");
end

// Release reset
RSTA = 0; RSTB = 0; RSTC = 0; RSTD = 0; RSTP = 0; RSTM = 0; RSTCARRYIN
= 0; RSTOPMODE = 0;

CEA = 1;
CEB = 1;
CEC = 1;
CED = 1;
CEP = 1;
CEM = 1;
CECARRYIN = 1;
CEOPMODE = 1;

//path 1:
A=20; B=10; C=350; D=25;

```

```

OPMODE=8'b11011101;
CARRYIN=$random; BCIN=$random; PCIN=$random;

repeat(4) @(negedge clk);

if(P !== 48'h32 || POUT !== 48'h32 || BCOUT !== 18'hf || M !== 36'h12c
|| CARRYOUTF !== 1'b0 || CARRYOUT !== 1'b0) begin
    $display("Test case 1 failed!");
    $display("Expected P=32, M=12c, BCOUT=f, CARRYOUT=0, CARRYOUTF=0");
    $display("Got P=%h, M=%h, BCOUT=%h, CARRYOUT=%b, CARRYOUTF=%b", P,
M, BCOUT, CARRYOUT, CARRYOUTF);
    $stop;
end else begin
    $display("Test case 1 passed.");
end

//path 2:
A=20; B=10; C=350; D=25;
OPMODE=8'b00010000;
CARRYIN=$random; BCIN=$random; PCIN=$random;

repeat(3) @(negedge clk);

if(P !== 48'h0 || POUT !== 48'h0 || BCOUT !== 18'h23 || M !== 36'h2bc
|| CARRYOUTF !== 1'b0 || CARRYOUT !== 1'b0) begin
    $display("Test case 2 failed!");
    $display("Expected P=0, M=2bc, BCOUT=23, CARRYOUT=0, CARRYOUTF=0");
    $display("Got P=%h, M=%h, BCOUT=%h, CARRYOUT=%b, CARRYOUTF=%b", P,
M, BCOUT, CARRYOUT, CARRYOUTF);
    $stop;
end else begin
    $display("Test case 2 passed.");
end

//path 3:
A=20; B=10; C=350; D=25;
OPMODE=8'b00001010;
CARRYIN=$random; BCIN=$random; PCIN=$random;

repeat(3) @(negedge clk);

if(P !== 48'h0 || POUT !== 48'h0 || BCOUT !== 18'ha || M !== 36'hc8 ||
CARRYOUTF !== 1'b0 || CARRYOUT !== 1'b0) begin
    $display("Test case 3 failed!");
    $display("Expected P=0, M=c8, BCOUT=a, CARRYOUT=0, CARRYOUTF=0");
    $display("Got P=%h, M=%h, BCOUT=%h, CARRYOUT=%b, CARRYOUTF=%b", P,
M, BCOUT, CARRYOUT, CARRYOUTF);

```

```

    $stop;
end else begin
    $display("Test case 3 passed.");
end

//path 4:
A=5; B=6; C=350; D=25;
OPMODE = 8'b10100111;
CARRYIN=$random; BCIN=$random; PCIN=3000;

repeat(3) @(negedge clk);

if(P !== 48'hfe6fffec0bb1 || POUT !== 48'hfe6fffec0bb1 || BCOUT !==
18'h6 || M !== 36'h1e || CARRYOUTF !== 1'b1 || CARRYOUT !== 1'b1) begin
    $display("Test case 4 failed!");
    $display("Expected P=fe6fffec0bb1, M=1e, BCOUT=6, CARRYOUT=1,
CARRYOUTF=1");
    $display("Got P=%h, M=%h, BCOUT=%h, CARRYOUT=%b, CARRYOUTF=%b", P,
M, BCOUT, CARRYOUT, CARRYOUTF);
    $stop;
end else begin
    $display("Test case 4 passed.");
end

$display("All test cases passed.");
$stop;
end
endmodule

```

3 DO.File

```

#create Work Folder
vlib work

#Compile files with names
vlog DSP.v REG_pipe.v DSP_tb.v

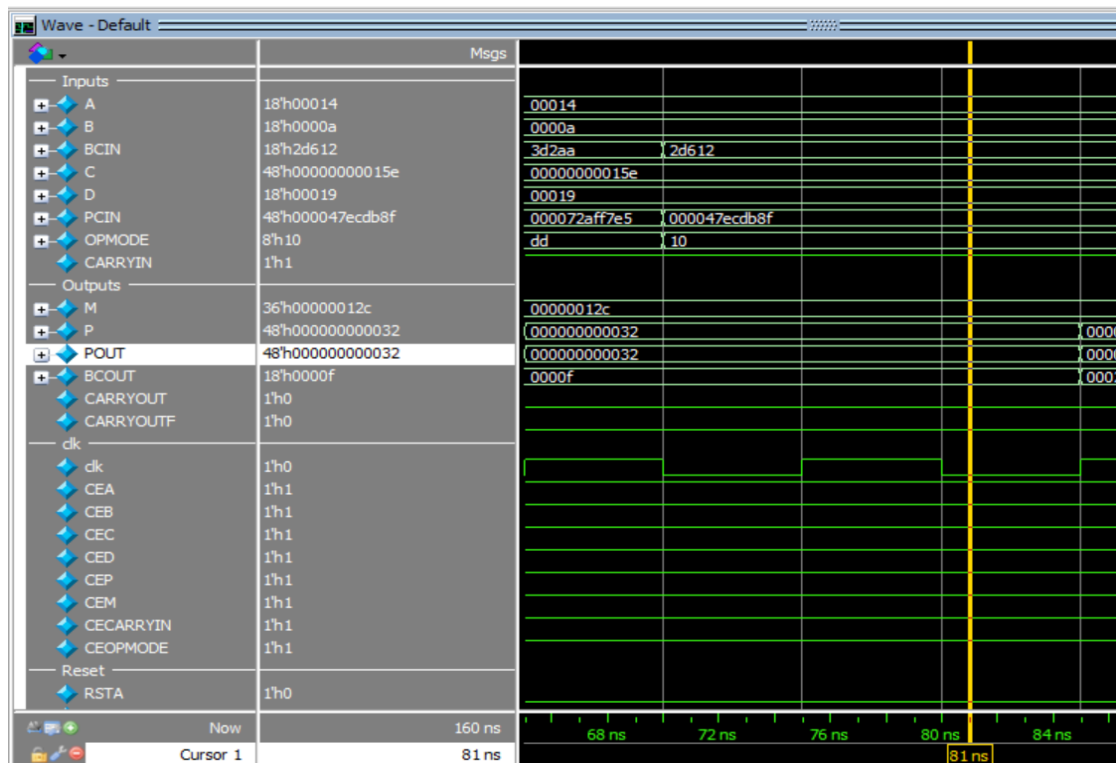
#simulate The TB file with module Name
vsim -voptargs=+acc work.DSP_tb

■ #add objects name to wave Window
■ #add wave -position insertpoint \sim:/tb/dut/*
add wave *
run -all
wave zoom full

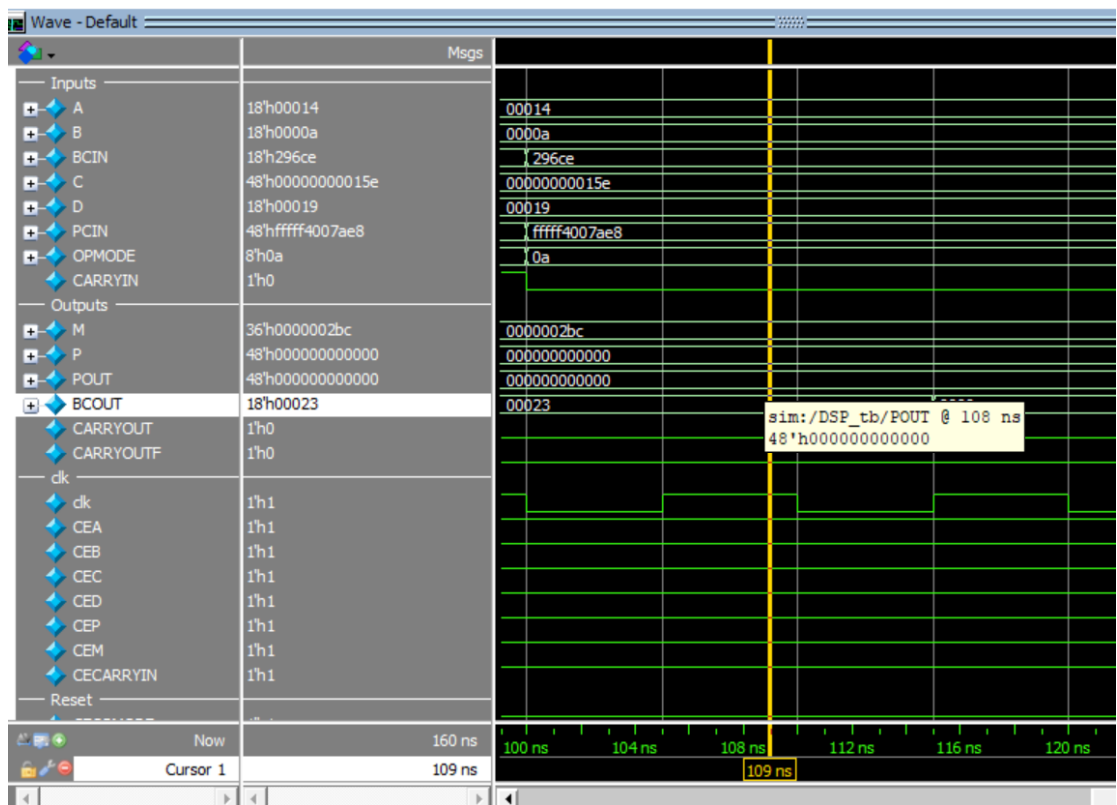
```

4 QuestaSim Snippets:

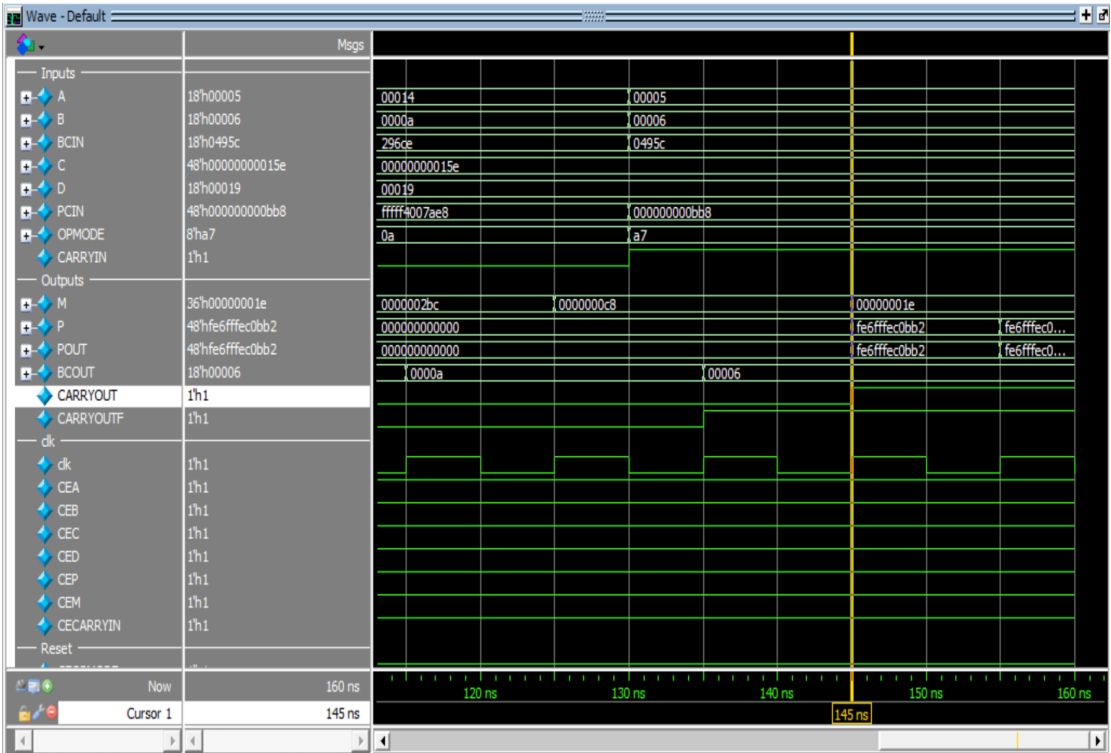
Path 1:



Path2



Path 3 & Path 4



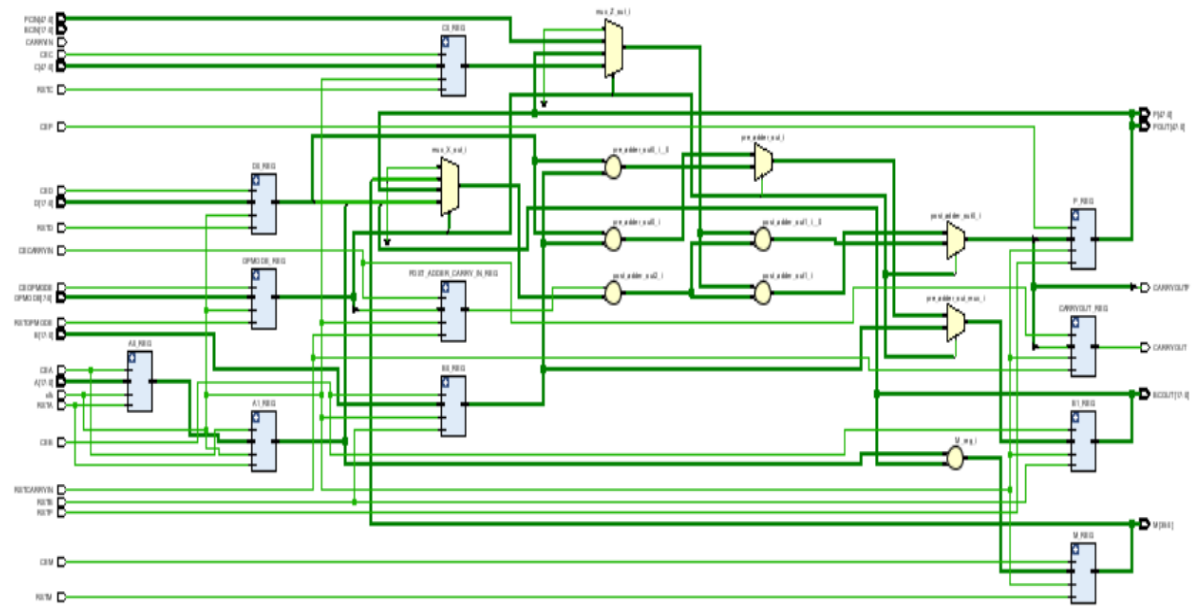
5 Constraint File:

```
@ dsp_constraints_spartan6.xdc
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in
5
6  ## Clock signal
7  set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports CLK]
8  create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports CLK]
9
10
11  ## Switches
```

```
## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCC0 [current_design]

## SPI configuration mode options for QSPI boot, can be used for all designs
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

6 Elaboration (“Messages” tab & Schematic snippets)

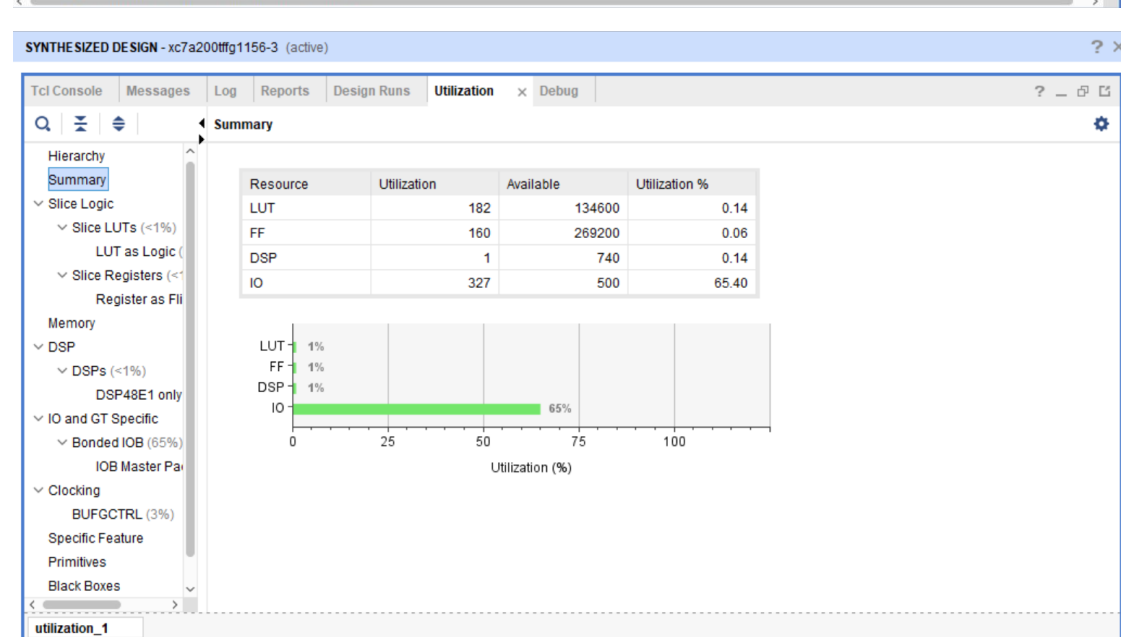


ELABORATED DESIGN - xc7a200tfg1156-3 (active)

Tcl Console Log Reports Design Runs Messages

Warning (22) Info (23) Status (11) Show All

- Vivado Commands (3 infos)
 - General Messages (3 infos)
 - [IP_Flow 19-234] Refreshing IP repositories
 - [IP_Flow 19-1704] No user IP repositories specified
 - [IP_Flow 19-2313] Loaded Vivado IP repository 'C:/Users/kerol/Desktop/folders/Vivado/Vivado/2018.2/data/ip'.
- Elaborated Design (22 warnings, 20 infos)
 - General Messages (22 warnings, 20 infos)
 - [Synth 8-6157] synthesizing module 'DSP' [DSP.v:1] (6 more like this)
 - [Synth 8-6155] done synthesizing module 'REG_pipe' (1#1) [REG_PIPE.v:1] (6 more like this)
 - [Synth 8-226] default block is never used [DSP.v:115] (1 more like this)
 - [Synth 8-3331] design REG_pipe__parameterized0 has unconnected port rst (21 more like this)
 - [Device 21-403] Loading part xc7a200tfg1156-3
 - [Project 1-570] Preparing netlist for logic optimization
 - [Opt 31-138] Pushed 0 inverter(s) to 0 load pin(s).
 - [Project 1-111] Unisim Transformation Summary:
No Unisim elements were transformed.



Component	Utilization (%)
LUT	1%
FF	1%
DSP	1%
IO	65%

SYNTHESIZED DESIGN - xc7a200tfg1156-3 (active)

Tcl ConsoleMessagesLogReportsDesign RunsTiming x Debug

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Check Timing (326)

Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

User Ignored Paths

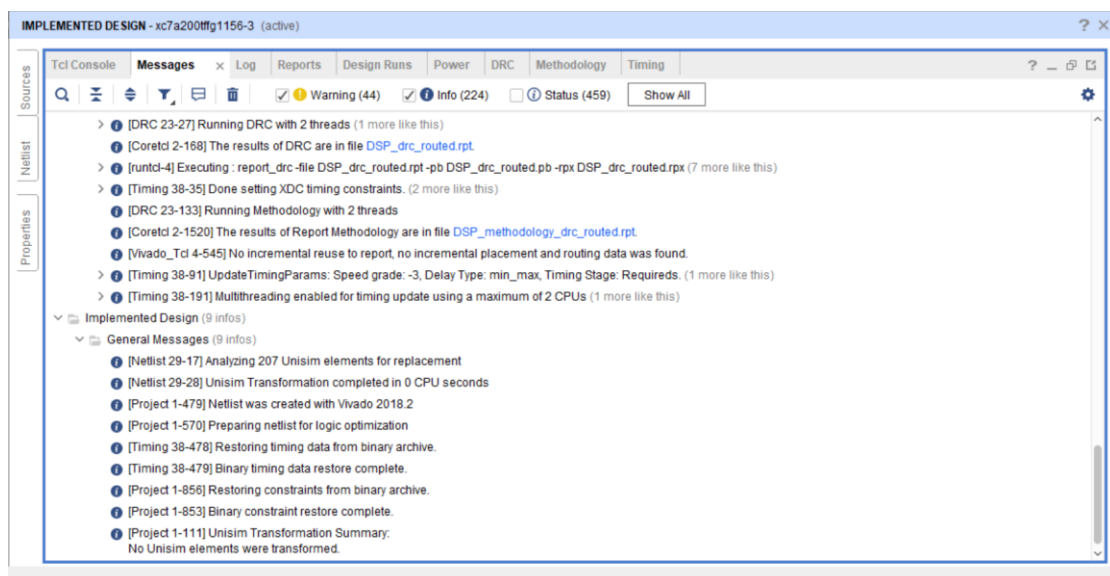
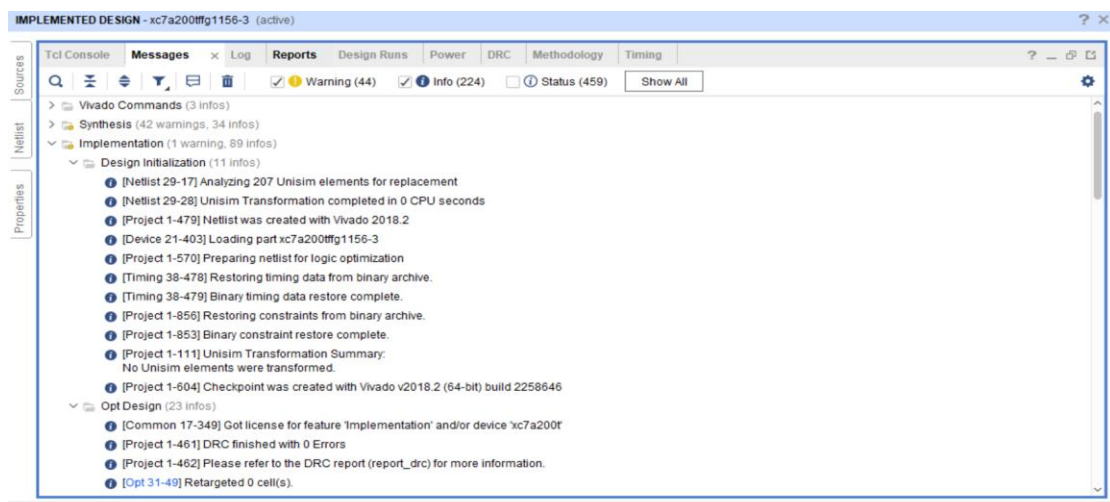
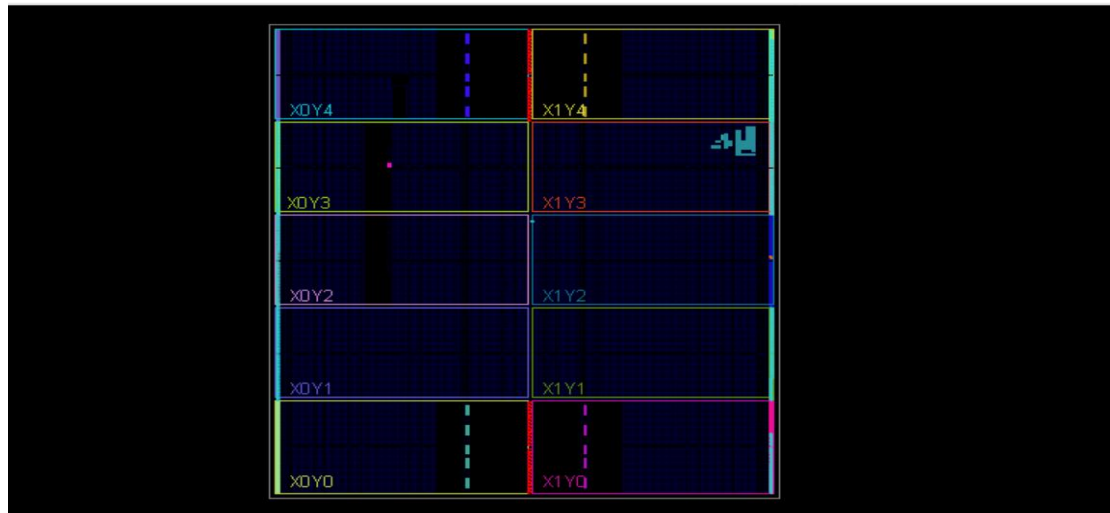
Unconstrained Paths

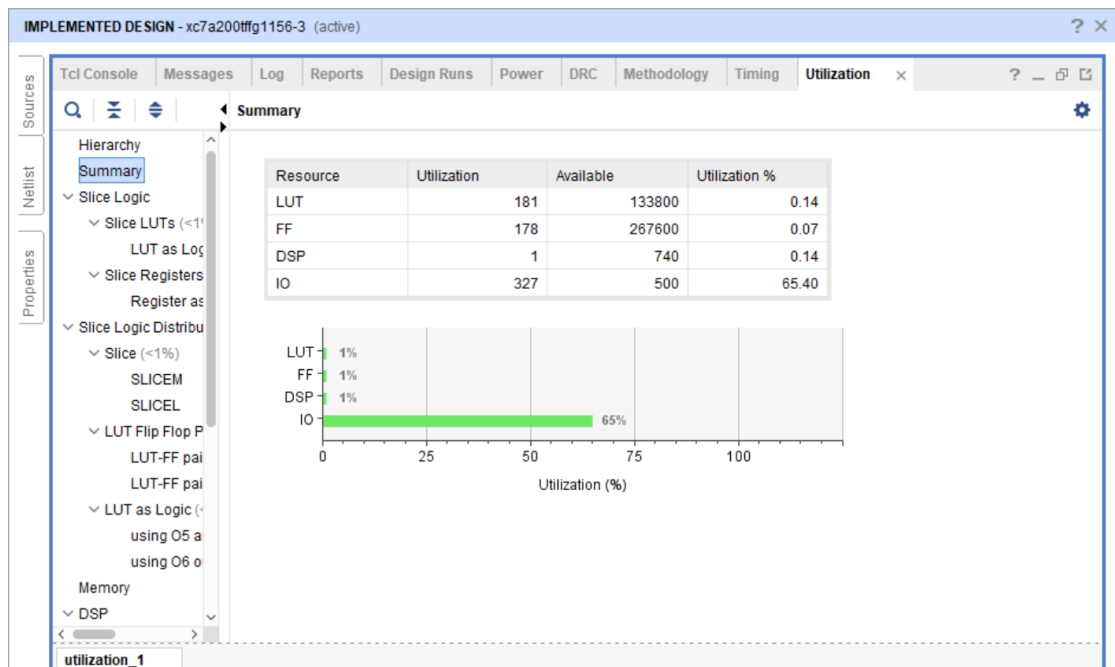
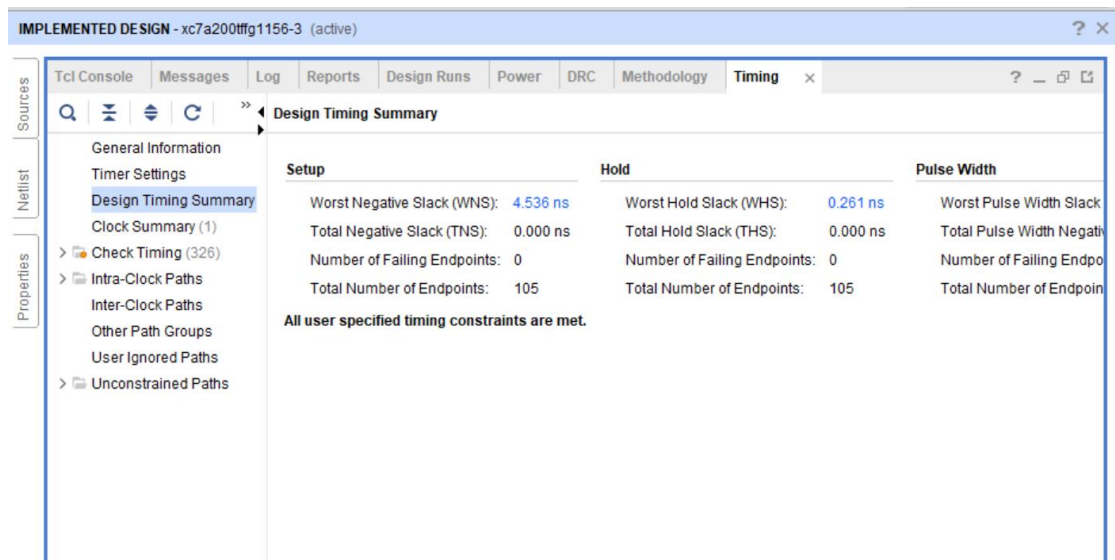
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.172 ns	Worst Hold Slack (WHS): 0.182 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 87	Total Number of Endpoints: 87	Total Number of Endpoints: 162

All user specified timing constraints are met.

Timing Summary - timing_1

8_ Implementation (“Messages” tab, Utilization report, timing report & device snippets)





9_Linting (snippets showing no errors)

Lint Checks

Filter: Type here

Waived

Fixed

Pending

Uninspected

Bug

Verified

Total : 1 Selected : 0

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
		multi_ports_in_singl...		Multiple ports are declared in one line. Mod...	DSP	Rtl Design ...	open	unass...	3.5.6.3